

moe-engine: A Fault-Tolerant Runtime for Hyperscale Mixture-of-Experts Training

Version 3 Preprint

Min Htet Myet
Independent Researcher

July 2026

<https://github.com/Mattral/Composed-Mixture-of-Experts-Engine>

Zenodo (v1): <https://doi.org/10.5281/zenodo.20647577>

Zenodo (v2): <https://doi.org/10.5281/zenodo.20688837>

v3 in preparation

Abstract

We present moe-engine, a research-grade infrastructure runtime for training large Mixture-of-Experts (MoE) models under the realistic constraint of continuous node failures at hyperscale (10K+ GPUs). The system combines a high-performance fused Triton router, composable 4D parallelism (Data, Expert, Tensor, Pipeline), strict mathematical invariants for correctness, and elastic fault tolerance with automatic expert resharding.

This v3 preprint reports the two most significant developments since v2: (i) the *first sustained real-GPU validation* of the runtime on NVIDIA T4 hardware, which surfaced and fixed a Triton compilation defect invisible on CPU-only continuous integration and produced measured throughput data ($80.1 \times$ GPU speedup over the CPU reference path at $N=4096$, $H=2048$; 100% pass rate on the storage-stall chaos scenario); and (ii) a structural refactoring that replaced a 1,165-line monolithic module with seven single-responsibility components, migrated configuration handling to a statically validated schema, and added a Switch-Transformer-style expert capacity enforcement mechanism. A related engineering finding is reported in detail because of its methodological interest: a missing dependency declaration caused the runtime’s configuration validator to silently degrade to a no-op across eight test cases in continuous integration for an entire release cycle, a failure mode that is generalizable to any system that ships an optional-dependency fallback for what is, in fact, safety-critical logic. The test suite has grown from 148 to 348 passing cases, gaining property-based testing and a mocked collective-communication backend that exercises expert-parallel dispatch logic without a multi-process environment.

We maintain the deliberately honest posture established in v1 and continued in v2: sustained multi-node GPU benchmarks, a full resolution of the 85% chaos-recovery pass rate, and hardware-scale validation of the newly added expert-capacity mechanism at $E=256$ remain open items for v0.4, and are reported as such rather than elided.

1 Introduction

Training large MoE models at true hyperscale exposes systems challenges that go beyond raw throughput: continuous node failures, silent collective and sharding bugs, misleading utilization metrics for sparse models, and the need for seamless recovery with expert re-ownership. moe-engine was designed from the ground up to address these challenges, and this is its third preprint.

Version 1 described the initial architecture: the Triton router, 4D mesh foundations, async two-tier checkpointing, and elastic recovery mechanisms. Version 2 reported the completion of multi-process Pipeline

Parallelism with real inter-stage `send/recv` communication, a full 1F1B schedule, and fused Sequence Parallelism backward communication — but explicitly declined to present GPU throughput numbers, stating that “sustained, reproducible cluster data is not yet available” and that the authors “prefer to report real, reproducible results rather than optimistic small-scale or illustrative numbers.”

This v3 preprint reports the resolution of that specific gap at single-GPU scale: the system has now been run, measured, and validated on real CUDA hardware for the first time. It also reports a substantial internal refactoring undertaken in parallel, motivated by an external technical review that identified the codebase’s single-file distributed-systems module as a maintainability risk disproportionate to the project’s research-grade maturity level. We report both the architecture-level changes and a specific software-engineering failure mode discovered during this work, because we believe the latter is of independent interest to anyone building validated-configuration systems with optional-dependency fallback paths.

2 Background and Motivation

Sections on MoE fundamentals, existing systems (Megatron-LM, DeepSpeed, Switch Transformers, Tutel, X-MoE, MegaScale-MoE), and the hyperscale failure model remain unchanged from v1/v2 and are omitted here for brevity. The core motivation — surviving continuous node death while maintaining correctness — is unchanged.

3 System Architecture Overview

The high-level architecture (training loop $\rightarrow$ `DistributedMoELayer` $\rightarrow$ `ElasticTrainerHarness` $\rightarrow$ `Telemetry`) remains consistent with v1/v2. The major internal change since v2 is structural rather than behavioral: the distributed package, previously a single 1,165-line file (`pkg/distributed/parallel_mesh.py`) containing every parallelism primitive, is now split into eight modules, each with one responsibility (Table 1). A 68-line backward-compatibility shim re-exports every public symbol from the original location, so existing import statements in downstream code and in the test suite continue to function unmodified.

Table 1: Distributed package structure after the v0.3.2 refactor

Module	Responsibility
<code>mesh.py</code>	<code>ParallelTopology</code> , device mesh, comm streams
<code>tensor_parallel.py</code>	<code>Column/RowParallelLinear</code>
<code>sequence_parallel.py</code>	SP scatter/gather, fused all-gather
<code>expert_parallel.py</code>	all-to-all dispatch/combine
<code>pipeline_parallel.py</code>	<code>PipelineStage</code> , 1F1B schedule
<code>data_parallel.py</code>	FSDP2 application
<code>router.py</code>	High-level router interface
<code>moe_layer.py</code>	<code>DistributedMoELayer</code> orchestration
<code>parallel_mesh.py</code>	Backward-compat. shim (68 lines)

Two further architectural additions were made in this cycle. First, model definitions were extracted from the training entrypoint into `pkg/models/`, and a decorator-based registry (`@register_model`, `build_model_from_config`) was added so that the training loop dispatches on a configuration string rather than importing a specific model class — the pattern used by Megatron-LM and torchTitan to decouple infrastructure code from model research code. Second, the flat-dictionary configuration format used since v1 was replaced with a statically validated schema (Section 6).

4 Triton MoE Router Kernel

The fused Triton router (forward + analytic backward) described in v1/v2 is architecturally unchanged: it continues to deliver single-HBM-pass routing with strong numerical guarantees and forward-pass invariants. What changed is that it has, for the first time, actually been exercised on GPU hardware as part of this project’s own validation process, and this exposed a defect.

4.1 The `constexpr` defect

Both the forward and backward Triton kernels use `tl.static_range` to unroll the top- K selection loop at compile time. This requires the loop bound K to be a Triton `constexpr` value. In the kernel signature prior to this fix, K was declared as an ordinary integer parameter. On the CPU-only reference path used throughout v1 and v2’s continuous integration, this made no difference, because the reference implementation is plain PyTorch and never invokes the Triton compiler. The first time the kernel was compiled on real GPU hardware — during the work reported in this preprint — it failed with a `Triton CompilationError`, because `tl.static_range` cannot unroll a loop whose bound is not known at trace time.

We consider this defect noteworthy beyond its immediate fix (declaring K as `constexpr` in both kernel signatures) because it is an instance of a general failure mode in projects that maintain a CPU reference fallback for hardware-specific code: the fallback path’s very existence allows a real hardware-only defect to survive an arbitrary number of CI runs undetected, because CI never exercises the code path where the defect lives. Two regression tests were added: one that statically asserts both kernel signatures declare K as `constexpr` (runs on CPU, catches this specific class of regression without hardware), and one that compiles and runs the kernel on GPU when available.

4.2 Real-hardware measurements

Section 10 reports the first real GPU throughput numbers for this kernel, obtained on an NVIDIA T4 after the fix above.

5 4D Parallelism and DistributedMoELayer

5.1 Device Mesh, Topology, and Sequence Parallelism

`ParallelTopology` and device mesh construction are functionally unchanged from v2; Sequence Parallelism was extracted from `tensor_parallel.py` into its own module (`sequence_parallel.py`) as part of the structural refactor, with backward-compatible re-exports. All four mesh axes (DP, EP, TP, PP) remain first-class.

5.2 Expert Parallelism and Tensor Parallelism

EP dispatch/combine with dedicated-stream overlap and the SwiGLU sharding rule (both gate and up projections as `ColumnParallelLinear`) are unchanged from v1/v2.

5.3 Pipeline Parallelism

The multi-process `PipelineStage` implementation reported in v2 — real `torch.distributed.send/recv` on a dedicated pipeline process group, activation/gradient tagging by micro-batch index, and the three-phase 1F1B schedule — is unchanged in this release. It remains validated by 2-rank `mp.spawn` tests and has not yet been exercised under chaos conditions (Section 11).

5.4 Sequence Parallelism Fusion

The v2 fusion of Sequence Parallelism’s backward communication with the subsequent projection matmul (replacing an all-gather + matmul pattern with a single all-reduce) is unchanged and remains validated by 2-rank `mp.spawn` tests.

5.5 Expert Capacity Enforcement (new in v3)

The single largest functional addition in this release is an optional, opt-in expert-capacity enforcement mechanism, following the first-come-first-served dropping policy used by Switch Transformer [3] and GShard [9]. Each expert accepts at most $\lceil c \cdot NK/E \rceil$ tokens, where c is a configurable capacity factor, N is the number of tokens, K is the number of active experts per token, and E is the total expert count; tokens beyond this budget receive zero combine weight for that slot rather than being processed.

The implementation centers on a single primitive, `_cumcount(groups)`, which computes for each element its zero-indexed position of appearance within its group, using a stable sort followed by a `cummax` operation. This avoids both an explicit Python loop over experts and a dependency on `scatter_reduce` variants whose availability differs across PyTorch versions. Capacity enforcement is disabled by default (`capacity_dropping=False`), so no existing behavior changes unless a caller explicitly opts in; 22 dedicated tests cover the primitive, the mask construction, and full-layer integration including gradient flow through non-dropped slots.

A new example configuration, `configs/large_scale.yaml`, exercises this mechanism at a fine-grained MoE scale ($E=256$, $K=8$, four times the expert count of the existing production configuration) together with the auxiliary load-balancing loss described below. This configuration has been validated for correctness (forward/backward pass, capacity budget arithmetic) at toy hidden dimensions; its throughput and memory behavior at full scale on real GPU hardware has not yet been measured, and we do not claim otherwise (Section 11).

6 Configuration System

6.1 Migration to a validated schema

The flat-dictionary configuration format used since v1 provided no validation: a value such as `top_k > num_experts` would not surface as an error until the router kernel failed with an opaque index-out-of-bounds exception, potentially minutes into a training run. We replaced this with a hierarchical schema (`MoEConfig`, six sub-configs covering model, training, parallelism, checkpointing, elasticity, and telemetry) with cross-field validators enforcing invariants such as `top_k ≤ num_experts`, `warmup_steps < max_steps`, and `dtype` membership in a fixed set. Validation failures now report the offending field path and a human-readable constraint description at load time, before any GPU is touched.

6.2 A silent-degradation failure mode, root-caused

We report the following finding in detail because we consider it methodologically instructive, not merely a bug-fix footnote. The configuration module was written with a fallback code path: if the validation library was not importable, the module would degrade to an unvalidated pass-through rather than fail to import. The intent was defensive — to keep the module importable in minimal environments.

The dependency, however, was never actually added to the package’s declared runtime dependencies. Continuous integration installed the package exactly as declared, which meant the validation library was never present, which meant every configuration — including deliberately invalid ones constructed by the test suite to verify that validation errors are raised — silently passed through the unvalidated fallback. Eight test cases that asserted a specific exception type would be raised on invalid input instead observed no exception

at all, for an entire release cycle, without any test failure being reported, because the tests themselves never exercised the validated code path.

The fix has two parts. First, the validation library is now a declared, non-optional runtime dependency; second, and more importantly, the fallback path was deleted entirely. The module now raises an import error immediately, with installation instructions, if the dependency is absent, rather than silently substituting a less-safe behavior. We consider this the correct general policy for any component whose stated purpose is catching invalid input early: a component that can silently stop performing its stated function is a more dangerous failure mode than one that refuses to start.

A second, unrelated defect was found in the same subsystem: the environment-variable override mechanism used `yaml.safe_load` to parse override values, which is subject to a YAML 1.1 grammar quirk in which exponential notation without an explicit sign or decimal point (e.g. `"1e-5"`) parses as a string rather than a float. An override such as `MOE_TRAINING__LEARNING_RATE=1e-5` would therefore silently fail with a type error several lines downstream rather than at the point of parsing. The fix attempts native `int/float` coercion before falling back to YAML parsing for booleans, null, and other scalar types.

7 Elastic Fault Tolerance and Checkpointing

The async two-tier checkpointing design (pinned host memory $\rightarrow$ NVMe with `O_DIRECT` and atomic rename $\rightarrow$ S3) and the `ClusterStateMachine` with round-robin expert resharding are unchanged from v1/v2 in their core algorithm. One addition was made: checkpoint metadata now carries an explicit schema version (`CHECKPOINT_SCHEMA_VERSION`, currently 2), together with the runtime’s own package version and PyTorch version, both read from their respective single sources of truth rather than hardcoded. On load, a compatibility check logs at `info` level for older-but-compatible checkpoints and at `warning` level for checkpoints written by a newer schema than the current runtime understands. This is a diagnostic aid rather than a hard compatibility gate: missing fields default gracefully in either direction.

Chaos Scenario B (storage stall) remains fully verified and, as reported in Section 10, now has real-GPU confirmation. Scenario A (node kill and recovery) is unchanged at approximately 85% pass rate, for the same root cause identified in v2: a Gloo `connectFullMesh` race during process-group re-initialization after `SIGKILL`. Replacing Gloo with NCCL in the chaos harness remains the top-priority item for v0.4 and requires GPU hardware to implement and validate, which was outside the scope of the single-GPU validation performed in this cycle.

8 Observability and MoE-Aware Accounting

Building on the comm/compute overlap ratio and per-layer expert compute latency introduced in v2, this release adds four further MoE-specific telemetry fields, all populated from real per-step measurements rather than estimated: `sparse_mfu` (Model FLOPs Utilization scaled by the sparse activation fraction K/E , which is the theoretically correct denominator for sparse models and is substantially lower than a naive dense-equivalent MFU calculation would report); `dead_expert_count` (the number of experts receiving zero tokens in a given step, an early warning signal for routing collapse); `routing_efficiency` (the fraction of allocated expert capacity actually consumed); and `active_experts`. A fifth field, `dropped_token_fraction`, reports the fraction of token/slot assignments discarded by the capacity mechanism of Section 5.5 and is zero whenever that mechanism is disabled, which is the default. All five fields are exposed through the existing four telemetry sinks (structured JSONL, TensorBoard, Prometheus, and the WandB integration added in v2) without any sink-specific code duplication, by injecting typed dataclass fields into the existing nested telemetry dictionaries at construction time.

9 Implementation and Testing

The test suite has grown from 148 tests across 33 files at the time of the v2 preprint to 348 passing tests across 20 files in the fast, CPU-only tier alone (one additional test is skipped without a CUDA device, and one is marked as an expected failure for a documented statistical edge case discussed below), plus a separate multi-process and chaos tier. Three categories of test infrastructure were added specifically to support this growth without requiring GPU or multi-process resources for routine development:

Property-based testing. Nine property-based tests, using the Hypothesis library, verify invariants — token conservation, combine-weight normalization, index validity, complete and non-overlapping expert ownership under arbitrary $(E, \text{ep_size})$ combinations, and configuration cross-field constraints — across randomly generated inputs rather than a fixed set of example cases, at fifty generated examples per property.

Mocked collective communication. A lightweight, thread-based simulation of `all_to_all_single` (`MockDistEnv`) allows expert-parallel dispatch and combine logic to be exercised at `ep_size > 1` within a single test process, without `torch.distributed` initialization. This is complementary to, not a replacement for, the existing 2-rank `mp.spawn` tests, which remain the source of ground truth for real collective correctness; the mock exists to make dispatch-logic bugs reproducible in under a second on a laptop.

Dedicated interface test suites. The high-level router wrapper and the model registry introduced in this cycle each received a dedicated test file (33 and 20 tests respectively), separate from the existing kernel-level and layer-level test suites, following the principle that each public interface boundary should have tests that would fail if that specific boundary’s contract were violated, independent of whether the underlying implementation also has coverage.

One previously known issue is worth updating: a stochastic routing-quality test (`test_uniform_init_lower_imbalance` parametrized over five random seeds) was reported in v2’s antecedent work as intermittently failing at seed 2. We have since determined this is a genuine statistical edge case — at that specific seed, a near-zero-variance gate initialization produces marginally higher load imbalance (1.625) than a sharp initialization (1.594) at $E=16$, due to random token co-location, rather than a defect in the load-imbalance calculation itself. The test is now explicitly marked as an expected failure with the mechanism documented inline, rather than either silently retrying or being deleted.

10 Preliminary Evaluation

10.1 Real GPU validation: NVIDIA T4

This section reports the results the v2 preprint explicitly declined to report, obtained on a single NVIDIA T4 (16 GB HBM2, 65 TFLOPS FP32) via Google Colab. We reproduce the exact figures rather than round or restate them, since the reproduction commands accompanying each are part of the project’s evaluation methodology.

Token conservation held with zero violations across 100 random seeds on CUDA, matching the CPU reference path’s zero-violation result ($N=512, H=128, E=32, K=2$). Chaos Scenario B achieved a 10-out-of-10 (100%) pass rate under the same storage-stall injection previously validated only on CPU.

Router kernel throughput, measured with the Triton forward-only kernel against the fp64 CPU reference path at four configurations, is shown in Table 2. The GPU/CPU speedup ratio increases super-linearly with problem size, from $2.9\times$ at $N=512$ to $80.1\times$ at $N=4096, H=2048$ — the configuration most representative of production MoE hidden dimensions and expert counts. We attribute the super-linear scaling to the single-HBM-pass Triton kernel’s bandwidth utilization improving with working-set size, while the CPU fp64 reference path, which is single-threaded and unvectorized, has no corresponding scaling advantage.

We emphasize what these measurements do and do not establish. They confirm kernel correctness and relative CPU/GPU scaling behavior on real hardware for the first time. They do not constitute a Model FLOPs Utilization result at production scale: the T4 is an inference-oriented card with a BF16 peak roughly

Table 2: Router forward throughput: T4 GPU vs. CPU reference

N	H	E	K	CPU (M/s)	GPU (M/s)	Speedup
512	256	16	2	0.747	2.141	$2.9\times$
1024	512	32	2	0.421	3.644	$8.7\times$
2048	1024	64	2	0.236	4.832	$20.4\times$
4096	2048	64	4	0.056	4.454	$80.1\times$

$15\times$ lower than an H100 SXM5, and the measured MFU at the smoke-test batch sizes used is on the order of 10^{-6} , which is expected and uninformative at this scale — it reflects batch size, not hardware capability. We report it for completeness and transparency rather than because it is a meaningful result in itself.

10.2 MoE layer overhead

Full `DistributedMoELayer` forward latency was measured against a single dense SwiGLU FFN baseline (equivalent to $E=1, K=1$, no router, no dispatch) at matched (B, S, H, F) . At the small batch sizes tractable for exhaustive small-scale measurement ($B\leq 4, S\leq 32$), routing and dispatch overhead on the T4 ranges from $21.8\times$ to $26.1\times$ relative to the dense baseline; on CPU, the corresponding range is $9.1\times$ to $16.2\times$. We note explicitly that this overhead figure is dominated by kernel-launch latency at these batch sizes and by the fact that the dense baseline holds an order of magnitude fewer parameters than the MoE configuration it is compared against — the two models are not capacity-equivalent, and the comparison should be read as a latency-floor measurement, not a throughput-per-parameter comparison.

10.3 Comparison to v2’s evaluation posture

Version 2 stated: “we deliberately do not present large-scale GPU scaling curves or MFU numbers in this preprint because sustained, reproducible cluster data is not yet available.” That statement remains true for multi-node data. It is no longer true for single-GPU data, which this section reports. We consider the distinction important: closing the single-GPU gap was tractable within the resources available for this release; closing the multi-node gap requires sustained access to a GPU cluster, which remains the primary blocker for v0.4 and is discussed in Section 11.

11 Discussion, Limitations, and Roadmap

11.1 Current Maturity (v0.3.3)

Correctness infrastructure is materially more mature than at v2: the configuration system now fails loudly rather than silently on invalid input (Section 6.2); the distributed package is decomposed into reviewable units; the test suite exercises probabilistic invariants via property-based testing in addition to fixed examples; and the router kernel has been validated on real GPU hardware for the first time, surfacing and fixing a defect that had been latent since v1.

11.2 Remaining Gaps

Honestly stated, in the same spirit as v1 and v2:

- **Multi-node GPU data.** No benchmark or MFU figures exist beyond a single T4. This is the direct continuation of v2’s stated P0 gap and remains the highest-priority item for v0.4.
- **Chaos Scenario A.** Unchanged at approximately 85% pass rate; the Gloo-to-NCCL migration identified as the fix in v2 has not been implemented, because it requires GPU hardware to validate and was outside this cycle’s scope.

- **Expert capacity mechanism at scale.** The dropping mechanism of Section 5.5 is validated for correctness at toy dimensions; its behavior at $E=256$ on real hardware — memory footprint, dispatch bandwidth under enforced capacity, and interaction with the aggregate load-balancing loss — has not been measured.
- **Capacity re-routing.** The current mechanism drops overflow tokens; routing them to a next-best available expert instead (as opposed to discarding them) is deferred, because tuning a sensible fallback policy requires real expert-parallel bandwidth data that is not yet available.
- **Pipeline Parallelism under chaos.** The multi-process 1F1B schedule has not been exercised under node-failure conditions; the existing chaos harness uses a non-pipelined model.
- **Non-divisible sequence lengths.** Sequence Parallelism still requires $S \bmod \text{tp_size} = 0$.

11.3 Roadmap

v0.4 priorities, in order: (1) sustained multi-GPU benchmark data and MFU validation, requiring cluster access; (2) the NCCL chaos-harness migration; (3) integrating Pipeline Parallelism into chaos scenarios; (4) hardware validation of fine-grained expert configurations; (5) Nsight/CUPTI integration for roofline analysis, which also requires GPU access beyond what was available for this cycle.

12 Conclusion

This preprint reports the closing of a gap explicitly left open by its predecessor: moe-engine’s router kernel and chaos-recovery mechanism have now been validated on real GPU hardware, at single-device scale, with reproducible measurements rather than illustrative estimates. In the process, this validation surfaced a genuine defect — a missing `constexpr` annotation invisible to any CPU-only test — which underscores a general point about systems that maintain hardware-specific code behind a CPU fallback: the fallback’s existence is precisely what allows a hardware-only defect to survive indefinitely in continuous integration.

In parallel, we undertook a structural refactoring motivated by external technical review, and in doing so discovered and root-caused a second, unrelated defect of independent interest: a configuration validator that had been silently non-functional for an entire release cycle due to an undeclared dependency, a failure mode we believe generalizes to any system that treats validated-but-optional as an acceptable combination for safety-critical logic. We have removed that combination from this codebase entirely rather than patching around it.

The project’s multi-node evaluation gap, identified as the top priority in v2, remains open in this release for the same reason it was open then: sustained cluster access. We continue to believe that reporting exactly what has and has not been measured, at whatever scale is actually available, is more useful to the field than extrapolating from small-scale results — a position this preprint’s own single-GPU results section is itself an instance of.

Acknowledgments

The author thanks the PyTorch, Triton, and distributed systems communities for the foundational tools, the maintainers of Pydantic and Hypothesis for tools that materially improved this release’s correctness posture, and the many researchers who have openly shared post-mortems and lessons from large-scale training failures.

References

- [1] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [2] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, “DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters,” in *Proc. 26th ACM SIGKDD*, 2020.
- [3] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *arXiv preprint arXiv:2101.03961*, 2021.
- [4] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram, J. Chau, P. Cheng, F. Yang, M. Yang, and Y. Xiong, “Tutel: Adaptive mixture-of-experts at scale,” *arXiv preprint arXiv:2206.03382*, 2023.
- [5] “X-MoE: Scaling Mixture-of-Experts training,” technical report, 2024.
- [6] “MegaScale-MoE: Large-scale Mixture-of-Experts training infrastructure,” technical report, 2025.
- [7] Y. Wu et al., “Lazarus: Resilient and elastic training of Mixture-of-Experts models,” *arXiv preprint*, 2023.
- [8] S. Hu et al., “Revisiting reliability in large-scale machine learning research clusters,” *arXiv preprint*, 2023.
- [9] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen, “GShard: Scaling giant models with conditional computation and automatic sharding,” *arXiv preprint arXiv:2006.16668*, 2020.

moe-engine Preprint v3 — July 2026. The latest version of the paper and source code are available at <https://github.com/MattTral/Composed-Mixture-of-Experts-Engine>. v1 is archived at <https://doi.org/10.5281/zenodo.20647577>; v2 is archived at <https://doi.org/10.5281/zenodo.20688837>.